

Laboratório 5

Alocação Dinâmica de Memória

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz
marcosleipnitz@ic.ufrj.br

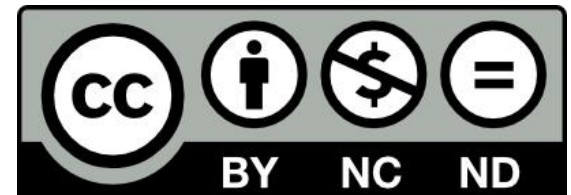


UFRJ

UNIVERSIDADE FEDERAL
DO RIO DE JANEIRO



INSTITUTO DE
COMPUTAÇÃO
UFRJ



Exercício 1

Escreva uma função **concatena** que tem o seguinte protótipo:

```
char *concatena(char *s1, char *s2);
```

A partir de duas cadeias de caracteres (acessadas a partir dos ponteiros *s1* e *s2*), a função deve criar uma **nova cadeia** de caracteres que é a **concatenação** da segunda cadeia ao final da primeira. Esta nova cadeia deve ser **alocada dinamicamente** pela função, cujo retorno seja um apontador para esta.

Faça um **programa principal** para testar a função implementada.

Exercício 2

Considere o programa principal a seguir e implemente as seguintes funções, necessárias para o seu adequado funcionamento:

```
int **aloca_linhas(int );  
void aloca_colunas (int **, int, int);  
void le_dados(int **, int, int );  
void imprime_matriz(int **, int, int );  
void troca_linhas (int **, int, int);
```

Exercício 2 (cont.)

- **aloca_linhas** deve alocar dinamicamente os ponteiros para as linhas da matriz.
- **aloca_colunas** deve alocar dinamicamente os elementos (colunas) de cada uma das linhas da matriz.
- **le_dados** deve solicitar ao usuário a entrada (a partir do teclado) de cada um dos elementos da matriz (cada coluna de cada linha da matriz).
- **imprime_matriz** deve exibir (em tela) cada um dos elementos da matriz (indicando linha e coluna correspondente ao dado)
- **troca_linhas** deve realizar a troca dos elementos da matriz, alterando os ponteiros das linhas e não substituindo os elementos em si.

Exercício 2 (cont.)



```
#include <stdio.h>
#include <stdlib.h>

int **aloca_linhas(int );
void aloca_colunas (int **, int, int);
void le_dados(int **, int, int );
void imprime_matriz(int **, int, int );
void troca_linhas (int **, int, int);

int main (void) {
    int **matriz;
    int lin, col;
    int linha1, linha2;
    puts("Qual o numero de linhas?");
    scanf("%d",&lin);
    matriz = aloca_linhas(lin);
    puts("Qual o numero de colunas?");
    scanf("%d",&col);
    printf("Alocando espa o para linhas.\n");
    aloca_colunas (matriz, lin, col);
    le_dados (matriz, lin, col);
    imprime_matriz (matriz, lin, col);
    puts("Qual a primeira linha a ser trocada?");
    scanf("%d",&linha1);
    puts("Qual a segunda linha a ser trocada?");
    scanf("%d",&linha2);
    troca_linhas (matriz, linha1, linha2);
    imprime_matriz(matriz, lin, col);
    return 0;
}
```

Exercício 3

Considere o programa principal a seguir e implemente as seguintes funções necessárias para o seu adequado funcionamento:

```
void * inicializa (Tfunc *, int);  
void * inclui_novos(Tfunc *, int, int);  
void imprime (Tfunc *, int, int);
```

Exercício 3 (cont.)

- **inicializa** deve alocar dinamicamente o vetor de estruturas e armazenar as informações de cada funcionário, solicitadas ao usuário (a partir do teclado).
- **inclui_novos** deve realocar dinamicamente o vetor de estruturas para armazenar as informações de mais usuários.
- **imprime** deve exibir (na tela) as informações dos funcionários no intervalo de posições indicado. Para mostrar as informações de todos os funcionários, o intervalo deve ser de **0** a **funcionários+adicionais**.

Exercício 3 (cont.)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct _func
{
    char nome[40];
    int ano_nasc;
    float renda;
} Tfunc ;

void * inicializa (Tfunc *, int);
void * inclui_novos(Tfunc *, int, int);
void imprime (Tfunc *, int, int);

int main(void)
{
    Tfunc *cadastro;
    int funcionarios, adicionais;
    printf("Quantos funcionarios? ");
    scanf("%d",&funcionarios);
    cadastro=inicializa(cadastro, funcionarios);
    if (!cadastro)
        return -1;
    printf("Quantos funcionarios a mais deseja adicionar? ");
    scanf("%d",&adicionais);
    cadastro = inclui_novos(cadastro, funcionarios, adicionais);
    if (!cadastro)
        return -1;
    printf("Lista de funcionarios cadastrados\n");
    imprime(cadastro, 0, funcionarios+adicionais);
    free(cadastro);
    return 0;
}
```

Desafio 4

Campo de Batalha - O Ataque Perfeito: Em um jogo de estratégia por turnos, o jogador comanda um exército em um campo de batalha representado por um tabuleiro retangular de dimensões $L \times C$, onde cada célula contém um número inteiro positivo representando o poder ofensivo acumulado das tropas inimigas posicionadas naquela região. Para planejar um ataque eficiente, o jogador pode escolher uma única zona de ataque retangular com dimensões $M \times N$ (ou seja, M linhas e N colunas), que representará a área em que suas forças lançarão uma ofensiva concentrada. As zonas de ataque são pré-definidas e não podem se sobrepor, ou seja, o campo de batalha é dividido logicamente em blocos $M \times N$, e cada célula pertence a exatamente um desses blocos. O objetivo do jogador é maximizar o impacto do ataque, atingindo a maior soma possível de poder ofensivo inimigo dentro de um único bloco de $M \times N$ células.

Tarefa: Escreva um programa que, dado um mapa do campo de batalha contendo o poder ofensivo das tropas inimigas em cada célula, determine qual é o maior valor total de poder ofensivo inimigo que pode ser atingido com um único ataque (zona de ataque).

Desafio 4 (cont.)

- **Entrada:** A entrada contém um único conjunto de testes que deve ser lido do dispositivo de entrada padrão. A primeira linha da entrada contém quatro números inteiros, **L**, **C**, **M** e **N**. **L** e **C** representam respectivamente o número de linhas ($L \geq 1$) e de colunas ($C \geq 1$) de células existentes no campo de batalha. **M** e **N** representam respectivamente o número de linhas ($1 \leq M \leq L$) e de colunas ($1 \leq N \leq C$) das zonas de ataque. As linhas contém **C** inteiros cada, representando o poder de ataque das tropas inimigas na célula localizada naquela linha e coluna. Note que L/M e C/N são sempre inteiros, para que não haja linha ou coluna de células de batalha que pertença a mais de uma zona de ataque.
- **Saída:** Seu programa deve imprimir na saída padrão uma única linha que contém o valor máximo de poder ofensivo das tropas inimigas que pode ser realizado em uma zona de ataque de **MxN** células.

Desafio 4 (cont.)

- Exemplos de entrada e saída:

Entrada 3 3 1 1 1 2 3 1 3 3 1 10 1 Saída 10	Entrada 4 4 2 1 1 2 3 4 5 6 7 8 1 10 5 2 1 5 9 10 Saída 15	Entrada 6 6 2 2 1 1 1 1 1 1 1 2 2 1 1 2 1 2 2 1 1 1 1 1 1 1 1 1 1 3 1 1 3 1 1 1 1 1 2 1 Saída 7
---	---	--

- Obs.:** seu programa deverá utilizar uma matriz **LxC**, **alocada dinamicamente**, para armazenar o poder de ataque do inimigo por célula no campo de batalha. O acesso à matriz deverá ser feito exclusivamente com **aritmética de ponteiros**.